Эффективная реализация лямбда-функций в объектно-ориентированных языках программирования

Болохононов Артем
Научный руководитель: Трепаков Иван Сергеевич

Анонимные функции

Определение

```
def filterEven(x: Int): Bool = {
  x % 2 == 0
}
```

Анонимные функции

Определение

- *Анонимные функции* (или *лямбда-функции*) — функции, которые создаются по ходу программы и не имеют уникального индентификатора,

```
val filerEvenLambda: Int ⇒ Bool = { x: Int ⇒ x % 2 }
```

Анонимные функции

Определение

- *Анонимные функции* (или *лямбда-функции*) — функции, которые создаются по ходу программы и не имеют уникального индентификатора,
- Анонимные функции могут замыкать окружающий контекст, в таком случае они называются *функциональными замыканиями*,

```
val filerEvenLambda: Int ⇒ Bool = { x: Int ⇒ x % 2 }
```

```
val capture: Int = 5
val functionalCaputre: ⇒ Int = { capture }

...

functionalCapture() // == 5
```

Анонимные функции

Определение

- *Анонимные функции* (или *лямбда-функции*) — функции, которые создаются по ходу программы и не имеют уникального индентификатора,

```
val filerEvenLambda: Int ⇒ Bool = { x: Int ⇒ x % 2 }
```

- Анонимные функции могут замыкать окружающий контекст, в таком случае они называются *функциональными замыканиями*,

```
val capture: Int = 5
val functionalCaputre: ⇒ Int = { capture }

...

functionalCapture() // == 5
```

- Сейчас в объектно-ориентированных языках программирования они используются повсеместно, часто используются при работе с коллекциями.

```
val collection: Seq[Int] = ...
collection.filter(x ⇒ x % 2)
collection.sort(x, y ⇒ x > y)
collection.foreach(x ⇒ println(x))
collection.map(x ⇒ x * x)
...
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами. Рассмотрим пример:

```
val lambda = { x ⇒ println(x) }
lambda(42)
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами. Рассмотрим пример:

```
class Lambda$$1 extends (Int ⇒ Unit) {
  override def apply(x: Int) = {
    println(x)
  }
}

val lambda = { x ⇒ println(x) }
lambda(42)
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами. Рассмотрим пример:

```
class Lambda$$1 extends (Int ⇒ Unit) {
  override def apply(x: Int) = {
    println(x)
  }
}

val lambda = new Lambda$$1()
lambda(42)
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами. Рассмотрим пример:

```
val capture = "arg = "

</d
val lambda = { x => println(capture + x) }
lambda(42)
```

Анонимные функции

В объектно-ориентированных языках программирования

В объектно-ориентированных языках лямбда-функции — это абстракция над синтетическими классами. Рассмотрим пример:

```
class Lambda$$1(val prefix: Str) extends (Int ⇒ Unit) {
  override def apply(x: Int) = {
    println(prefix + x)
  }
}

val capture = "arg = "
val lambda = new Lambda$$1(capture)
lambda(42)
```

Постановка задачи

Проблема

Каждая из анонимных функций в условиях *JVM* (*Java Virtual Machine*) — это полноценный объект, который требует как аллокации, так и сборки мусора. Из-за чего возникают издержки по памяти и времени.

Постановка задачи

Проблема

Каждая из анонимных функций в условиях *JVM* (*Java Virtual Machine*) — это полноценный объект, который требует как аллокации, так и сборки мусора. Из-за чего возникают издержки по памяти и времени.

Безусловно, существует набор оптимизаций, которые могут помочь в избавлении от создания объектов в куче.

Оптимизации

Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = SomeObject(42, 42.0, "42")
```

Оптимизации

Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

val obj = SomeObject(42, 42.0, "42")
useField(obj.a)
useField(obj.b)
useField(obj.c)
```

Оптимизации

Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
useField(objC)
```

Оптимизации

Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
useField(objC)

useObject(obj)
```

Оптимизации

Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
useField(objC)

useObject(rematerialize(objA, objB, objC))
```

Оптимизации

Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
useField(objC)

useObject(rematerialize(objA, objB, objC))
```

```
def useObject(x: SomeObject) = {
  globalField = x
  otherObject.x = x
  array.add(x)
}
```

Оптимизации

Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

val (objA, objB, objC) = (42, 42.0, "42")
useField(objA)
useField(objB)
useField(objC)

useObject(rematerialize(objA, objB, objC))
```

```
def useObject(x: SomeObject) = {
  globalField = x
  otherObject.x = x
  array.add(x)
}
```

```
def useObject(x: SomeObject)  // ???
```

Оптимизации

Анализ утеканий

```
class SomeObject(var a: Int, var b: Double, c: String)

{
  val (objA, objB, objC) = (42, 42.0, "42")
  useField(objA)
  useField(objB)
  useField(objC)

  useObject(rematerialize(objA, objB, objC))
  return obj  // ?
}
```

```
def useObject(x: SomeObject) = {
  globalField = x
  otherObject.x = x
  array.add(x)
}
```

```
def useObject(x: SomeObject)  // ???
```

Тестовый стенд

Huawei VM

Проблематика

Цель и задачи

Цель: разработать и реализовать оптимизацию лямбда-функций на основе анализа утеканий, которая уменьшит потребление памяти за счет увеличения количества размещений анонимных функций на стек,

Проблематика

Цель и задачи

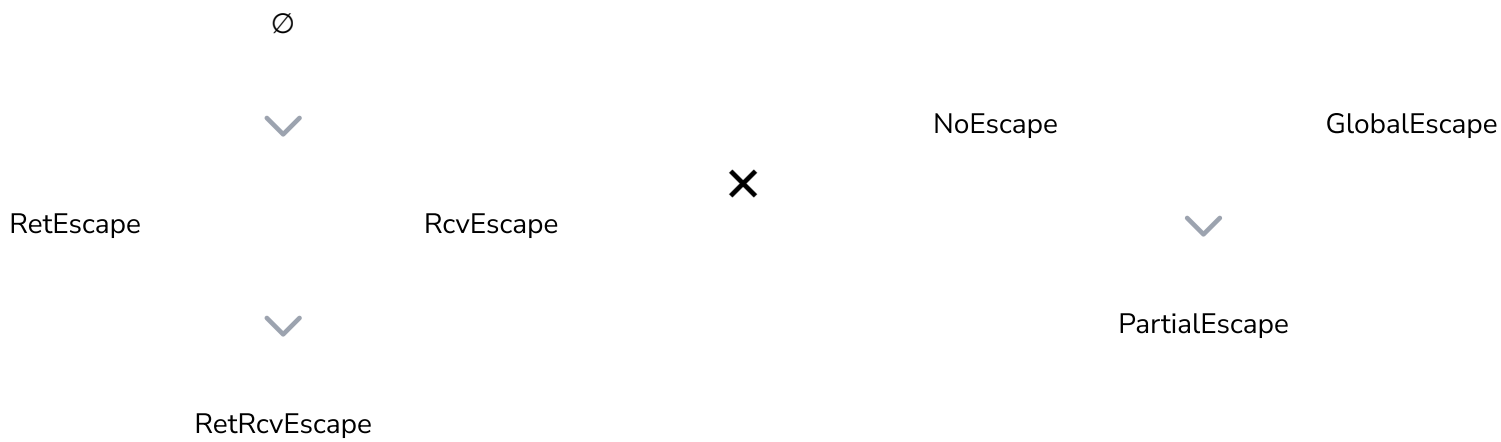
Цель: разработать и реализовать оптимизацию лямбда-функций на основе анализа утеканий, которая уменьшит потребление памяти за счет увеличения количества размещений анонимных функций на стек,

Задачи:

1. Изучить существующие подходы
2. Изучить текущую реализацию в компиляторе **Huawei VM**
3. Разработать и реализовать оптимизацию
4. Протестировать и замерить результаты

Анализ утеканий

Алгоритм

Анализ утеканий

Алгоритм